

Phase III Look Ahead

Project Name: Roommate Dashboard

Document ID: PhaseIII_LookAhead

Control Label: D0-v1-r1

Date: 4/12/2026

Prepared By: Group 6

Overview

The following summary describes how the Roommate Dashboard output from Phase II satisfies each of the evaluation areas that will be assessed in Phase III. Each area is addressed in relation to the work completed — code, documentation, testing, and process artifacts — delivered across both phases.

Evaluation Area	How Our Output Satisfied It
Features	The system delivers all core stakeholder-driven features: expense tracking with equal and custom bill splitting, chore assignment with automatic weekly rotation, a shared grocery list with purchase status tracking, direct roommate-to-roommate payment logging, and a real-time activity audit log. Every feature traces directly to a stakeholder need documented in Phase I.
Software Design	The application follows a layered MVC architecture enforced by Spring MVC, with a dedicated Service layer for business logic and a Repository layer for data access. This separation ensures maintainability and scalability. The design adheres to all five SOLID principles, as documented in the Phase II OOP implementation report.
Unit Testing	Integration tests in DashboardWorkflowTests cover all core workflows: expense creation, split payment marking, chore assignment and completion, grocery item purchasing, payment logging, activity log capture, and balance calculation. RoomatedashApplicationTests confirms the application context loads without error.
Code Reviews	The codebase is organized into clearly separated packages (model, service, controller, repo, dto, config, exception), making peer review straightforward. Consistent naming conventions, single-responsibility classes, and constructor injection throughout make the code easy to read, evaluate, and extend.
Coding Convention	The project follows standard Java and Spring Boot conventions: package-by-layer organization, PascalCase class names, camelCase

Evaluation Area	How Our Output Satisfied It
	method and field names, and consistent use of Spring annotations (@Entity, @Service, @Repository, @Controller, @Transactional). DTOs are clearly separated from domain entities.
Bug Management	Errors and edge cases are handled centrally through GlobalExceptionHandler, which catches DashboardException (business-rule violations) and ResourceNotFoundException (missing entities) and renders a user-facing error page. This prevents unhandled exceptions from surfacing to end users and simplifies debugging.
Documentation	Documentation was maintained consistently across both phases, covering requirements (inception, elicitation, non-functional), stakeholder analysis, user stories, use case diagrams, a user guide, a testing plan, sprint minutes, and the Phase II OOP implementation summary. All artifacts follow the team's document control naming convention.
Product Backlog	Features were derived from stakeholder needs and prioritized as a conceptual backlog. Core financial features (expense tracking, balance visibility, payment logging) were treated as highest priority, followed by household management, chore tracking, and grocery management — reflecting the stakeholder feedback gathered during elicitation.
Sprint Planning	Development was approached incrementally. The first sprint established the domain model, household management, and expense tracking. Subsequent work expanded into chore rotation, grocery management, and the activity log. This incremental approach is evidenced by the sprint meeting minutes documented in SprintMinute_D0-v1-r1.pdf and SprintMinute_D0-v2-r1.pdf.
Collaboration	Team members contributed across backend development, requirements analysis, testing, and documentation. Responsibilities were distributed so that no single area depended on one person, ensuring the deliverable remained complete and internally consistent across both phases.
Scrum Meetings	Regular check-ins were held to track progress, surface blockers, and ensure alignment between implementation and documented requirements. Meeting outcomes were recorded in sprint minute documents submitted as part of the Phase I artifact package and carried forward into Phase II.

Phase III Readiness

Based on the work completed in Phases I and II, the team is prepared to enter Phase III with a functioning application, a complete documentation package, and a traceable connection between stakeholder needs and implementation. The primary focus for Phase III will be refining the user interface, expanding test coverage, and addressing any remaining non-functional requirements such as security and session management.